

EYE OF ODIN: 3D Campus Lost & Found Tracker

System Architecture, Multi-Modal AI Engine & Comprehensive Output Documentation

Project Title:	Eye of Odin: Lost & Found 3D Tracker	Repository:	7Hansika7 / DBS-DSE-PROJECT
Tech Stack:	Node.js (Native REST) + Three.js WebGL	Target Domain:	University Campus Safety & Recovery
Dependencies:	Zero external npm packages (Pure JS)	Documentation:	Release v1.0 (Academic Report)

1. Executive Summary & Problem Formulation

University campuses are dense, high-traffic ecosystems where thousands of students, faculty, and visitors lose essential personal items daily—ranging from standard articles (water bottles, umbrellas) to high-consequence belongings (academic thesis laptops, research data drives, passports, prescription glasses, and vehicle keys).

Traditional campus lost-and-found desks face major operational bottlenecks: disjointed bulletin boards, poor communication between disparate campus departments, lack of urgency triage, and high vulnerability to fraudulent ownership claims. **Eye of Odin** solves these challenges by combining an interactive **Three.js 3D WebGL aperture interface** with an intelligent **5-Vector Multi-Modal AI Matching Engine**, an **Anti-Scam Claim Verification Protocol**, and an integrated **Campus Security Custody Locker Management System**.

Core Architecture Advantage: Eye of Odin operates with **zero external npm dependencies** on the backend, ensuring instant portable deployment, zero-latency startup on Node.js, and an ultra-lightweight client-side 3D rendering pipeline capable of running smoothly on all standard university lab computers and mobile browsers.

2. System Architecture & Component Design

The project is partitioned into distinct modular layers ensuring clean separation of concerns, persistent state tracking, and fast query execution:

Tier / Module	Technologies	Functional Responsibilities
Frontend 3D HUD Dock	Three.js, WebGL, Vanilla ES6+, CSS Glassmorphism	Interactive 3D mechanical iris, orbital item meshes, live telemetry cards, scrollytelling dock, printable QR poster generator, Web Audio procedural SFX.
Backend REST Server	Node.js Native HTTP, FileSystem, URL Modules	Zero-dependency microservice hosting static web assets, serving REST endpoints (/api/items, /api/stats, /api/claims), and handling CORS preflights.
Persistence Layer	JSON Flat-File Database (items.json, claims.json)	ACID-like safe file write transactions, audit logging, item status lifecycle tracking (active, in_vault, reunited).
AI Matching Engine	Vector Affinity Matrix & Jaccard Tokenizer	5-dimensional similarity computation matching newly reported items against existing database records with confidence percentages.

3. Multi-Modal AI Similarity Matching Engine

To minimize search friction and automate pairing between lost reports and found items, the engine calculates a composite affinity score using five weighted dimensions:

Vector Dimension	Weight	Algorithm & Computation Principle
Category Match	30%	Exact taxonomic equality + fallback to custom subcategories (Electronics, Books, ID Cards, Valuables).
Keywords & Semantics	25%	Stop-word filtered tokenization with Jaccard coefficient overlap computed across title, description, and metadata tags.
Spatial Telemetry	20%	Campus building coordinate proximity using Euclidean distance over 3D campus coordinates ($\sqrt{dx^2 + dz^2}$).
Color Affinity	15%	Hexadecimal RGB vector color delta distance combined with semantic color label matching.
Temporal Window	10%	Exponential temporal decay scoring items reported within close proximity of the incident timestamp.

4. Detailed Output Description & Interface Walkthrough

This section documents the visual, interactive, and functional outputs produced by the system across each module and state of execution:

Output 1: 3D WebGL Holographic HUD & Odin Eye

When launched, the browser renders an interactive 3D scene in the right viewport. A mechanical iris aperture pupil expands and contracts in response to mouse movement and scroll depth. Holographic concentric rings with runes rotate along the Z-axis, while five detailed 3D models (MacBook Pro, AirPods Pro, Brass Keyring, Student ID Badge, and Hydro Flask) orbit the central core. Hovering or clicking an item initiates orbital focus, displaying item telemetry.

Output 2: Priority-Coded Item Stream & Left HUD Dock

The left side displays a glassmorphic dashboard with live item counts (Lost, Found, Reunited, Vault Custody) and recovery rate metrics (e.g., 91% Success Rate). Items are tagged with distinct priority badges:

- CRITICAL (Red): Academic thesis laptops, passports, prescription glasses.
- HIGH (Amber): Smartphones, keychains, transit passes.
- STANDARD (Blue): Apparel, umbrellas, bottles, textbooks.

Filters allow instant isolation by status, category, urgency, and keyword search.

Output 3: Cinema View Mode (Unobstructed Viewport)

Clicking the '[Cinema View]' toggle smoothly collapses all HUD sidebars, overlays, and filter pills, providing an unobstructed 3D canvas for interactive exploration, presentation mode, or campus digital signage kiosk deployment.

Output 4: Multi-Step Lost & Found Reporting Wizard

Users trigger a modal wizard to register an item. Outputs include:

- Dynamic category selection with write-in custom tags.
- Interactive campus map selector assigning building ID and specific room/spot coordinates.
- Color swatch picker with automated semantic color labeling.
- Secret Ownership Question prompt to establish anti-scam safeguards before publishing.

Output 5: Anti-Scam Claim Verification Protocol

When a student clicks 'Claim Item', the system presents an ownership challenge (e.g., 'What sticker is on the laptop lid?' or 'Specify the last 4 digits of the serial ID'). The claimant's answer is recorded in claims.json, and contact info remains protected until the finder or security officer validates the response.

Output 6: Campus Security Custody Locker Manager

For valuable or sensitive items handed into campus safety, the Admin Portal outputs a locker management matrix (e.g., Locker A-04, Locker B-12). Staff can check-in items, update custody status to 'in_vault', log student verification IDs, and mark items as 'reunited' upon verified pickup.

Output 7: Printable High-Resolution QR Poster Generator

Clicking 'Generate Poster' dynamically formats a printable A4/Letter campus lost-and-found flyer. The output contains high-contrast typography, item specifications, location summary, and an auto-generated QR code linking directly to the digital claim modal.

5. REST API Specification & Data Model

The backend implements zero-dependency endpoints communicating strictly in UTF-8 JSON with automated CORS support:

Endpoint	Method	Parameters / Body	Response Payload Description
/api/health	GET	None	Service status, uptime seconds, server timestamp.
/api/stats	GET	None	Total counts (lost, found, reunited, in_vault) and recovery rate %.
/api/items	GET	?type=&category;=&priority;=&q;=	Array of filtered item objects matching query criteria.
/api/items	POST	{ title, type, priority, category, ... }	Newly created item record with generated UUID timestamp.
/api/items/:id	PATCH	{ status, custody, lockerId }	Updated item record reflecting modified state.
/api/claims	POST	{ itemId, claimantName, answer }	Confirmation receipt and pending verification ID.
/api/items/match/:id	GET	Item ID in URL path	Ranked matches sorted by 5-vector composite similarity score.

6. Execution, Verification & Testing Instructions

- 1. Launch Backend Server:**
cd CODE/backend && node server.js
Server will listen on **http://localhost:5050** (or custom \$PORT)

2. Access Frontend Application:
Open your browser and navigate to **http://localhost:5050**.
The 3D WebGL viewport will initialize automatically alongside the live HUD dock.

3. Test API Health & Stats:
curl http://localhost:5050/api/health
curl http://localhost:5050/api/stats

Summary & Academic Value: Eye of Odin demonstrates how modern WebGL graphics, spatial 3D interaction design, lightweight native backend architectures, and intelligent affinity scoring algorithms combine to transform campus administrative utilities into engaging, secure, and user-centric student services.